

Relazione progetto MOBD

DI MIRKO DAMIANO (0378007)

Università degli studi Roma “Tor Vergata”
Corso di Laurea Magistrale *Ingegneria dell’Automazione*

Indice

1	Introduzione	2
1.1	Air Quality	2
1.2	Heart Failure Clinical Records	2
2	Struttura del codice	2
2.1	Preprocessing dei dati	2
2.2	K-fold Cross-validation	3
2.3	Rete neurale	3
2.4	Fase di addestramento	4
3	Risultati e valutazione del modello	4
3.1	Air Quality	5
3.2	Heart Failure Clinical Records	6
4	Istruzioni d’uso	7
5	Bibliografia	8

1 Introduzione

Il progetto di Metodi di Ottimizzazione per Big Data prevede la realizzazione di un modello di machine learning atto alla risoluzione di un problema di **regressione**.

Sono state scelte due tipologie di dataset per la valutazione del modello:

1. *Air Quality*
2. *Heart Failure Clinical Records*

1.1 Air Quality

Si ha una raccolta temporale di dati derivante dalle rilevazioni sul campo di un dispositivo chimico, integrato con 5 sensori a ossidi metallici; quest'ultimo è stato posizionato in un'area ad alto tasso di inquinamento all'interno di una città italiana, registrando:

- 9358 campioni
- 15 feature

L'obiettivo principale consiste nell'addestrare la rete neurale a stimare in modo continuo la concentrazione di **Monossido di Carbonio** $CO(GT)$, espressa in mg/m^3 , presente nell'aria.

Qui il link al dataset utilizzato: [AirQuality](#)

1.2 Heart Failure Clinical Records

Si ha una raccolta di dati sanitari e clinici derivante dai cartellini di 299 pazienti affetti da insufficienza cardiaca, durante il periodo di monitoraggio a seguito di una diagnosi o di una terapia. In particolare tale dataset riassume il profilo fisiologico di ciascun paziente attraverso 12 feature cliniche, tra cui: età, livelli di creatinina, diabete, ipertensione e fumo.

La rete neurale deve imparare a stimare in modo continuo la durata del periodo di osservazione del paziente espresso in giorni.

Qui il link al dataset utilizzato: [HeartFailureClinicalRecords](#)

2 Struttura del codice

Inizialmente i dataset sono stati sottoposti ad una fase di preprocessing, in quanto presentavano colonne non rilevanti all'addestramento del modello; in particolare per il dataset Air Quality sono state eliminate le colonne relative a `Date` e `Time` e quelle contenenti solo valori `Nan`.

2.1 Preprocessing dei dati

Si è deciso infatti di:

- separare feature e label selezionando i valori della colonna target:

```
X = dataset.drop(columns = [target_column]).values #Matrice di features
y = dataset[target_column].values.reshape(-1, 1) #Vettore di label
```

- mescolare i dati per evitare che il modello impari sequenze temporali specifiche e i set siano bilanciati, in quanto i dataset presentavano un determinato ordinamento;

- dedicare l'80% dei dati al training set e il 20% al test set;
- standardizzare i dati poichè le variabili presenti nei dataset sono misurate con unità di misura e ordini di grandezza differenti

2.2 K-fold Cross-validation

Al fine di individuare la miglior configurazione della rete neurale, la selezione del numero di strati nascosti, dei neuroni al loro interno e del parametro λ per la regolarizzazione, è stata adottata una strategia di **K-fold Cross-validation**, con parametro $k=5$.

In particolare, nel file `config.py`, sono state definite due liste di parametri:

```
HIDDEN_LAYER_NEURONS_LIST = [12, 16, 32]
```

```
LAMBDA_LIST = [0, 1e-3, 0.1]
```

in modo tale da testare tutte le combinazioni possibili, alternando ad ogni iterazione il blocco di validazione relativo.

L'errore medio di validazione sui 5 fold è stato poi utilizzato come metrica di decisione per scegliere la combinazione ottimale, con il seguente esempio di output:

```
Miglior configurazione trovata:
```

```
Strati nascosti = [12, 16]
```

```
Lambda = 0.1
```

2.3 Rete neurale

Il modulo `dynamic_network.py` contiene la classe `NeuralNetwork`, progettata in modo tale da supportare tecniche di regolarizzazione e ottimizzazione.

Osservazione 1. A scopo informativo, il modulo `static_network.py` è presente nell'insieme di codici ma non viene utilizzato esplicitamente in quanto esso è stato creato preventivamente come modulo di verifica nel caso in cui la rete avesse un numero fissato di strati e neuroni interni

In primo luogo si utilizza l'*inizializzazione di He* per impostare i pesi W^ℓ di ogni strato nascosto ℓ , estraendoli da una distribuzione normale centrata in zero con una specifica deviazione standard:

$$W^\ell \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\ell-1}}}\right)$$

mentre i vettori di bias b^ℓ vengono inizializzati a zero per ciascuno strato.

Si ha poi una funzione di **forward propagation**, ove i dati fluiscono attraverso gli strati calcolando la combinazione lineare degli input:

$$z^\ell = W^\ell a^{\ell-1} + b^\ell$$

a cui si applica la funzione di attivazione relativa:

$$a^\ell = \varphi(z^\ell)$$

Osservazione 2. In particolare l'utente ha la possibilità di scegliere tra due funzioni di attivazione:

1. **ReLU**: $\varphi(z) = \max\{0, z\}$;

2. **Tangente Iperbolica**: $\varphi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1, 1)$

Trattandosi di un problema di regressione, per lo strato di output, viene utilizzata l'attivazione **lineare** $\varphi(z) = z$.

Tali funzioni e le corrispondenti derivate sono state definite nel file `functions.py`

La funzione di **back propagation** si occupa invece di calcolare i gradienti della funzione di errore:

$$E = \frac{1}{2} \|\varphi - y\|^2 + \lambda(\text{termine di regolarizzazione})$$

ciclando a ritroso su tutti gli strati e utilizzando la regola della catena:

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial z} \frac{\partial z}{\partial w}$$

Osservazione 3. Anche la scelta della regolarizzazione è lasciata all'utente, tra:

1. **Regolarizzazione L2**: $\|W\|_2^2$;
2. **Regolarizzazione L1 (LASSO)**: $\|W\|_1$

2.4 Fase di addestramento

L'addestramento della rete avviene tramite l'algoritmo **Mini-batch Stochastic Gradient Descent**, per cui nell'aggiornamento dei pesi-bias si utilizza una stima stocastica del gradiente $\tilde{\nabla} E$ e la si utilizza come direzione di discesa per la line search:

$$W^\ell \leftarrow W^\ell - \alpha \tilde{\nabla}_{W^\ell} E$$

$$b^\ell \leftarrow b^\ell - \alpha \tilde{\nabla}_{b^\ell} E$$

Nello specifico il numero totale di mini-batch viene calcolato mediante il numero di campioni `m` e la dimensione scelta, data dal parametro `MINI_BATCH_SIZE`.

All'inizio di ogni epoca il codice genera poi una permutazione casuale di tutti gli indici del training set così da assicurare che la struttura dei blocchi `X_batch` e `y_batch`, calcolati rispettivamente sugli indici del batch corrente, vari continuamente.

Sulla base di quest'ultimi si applicano infine le tecniche di ottimizzazione presentate precedentemente.

3 Risultati e valutazione del modello

Per la valutazione del modello sono state utilizzate principalmente tre tipologie di metriche:

- **MSE**: Scarto Quadratico Medio:

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (\varphi_i - y_i)^2$$

in fase di addestramento e cross-validation;

- **MAE**: Errore Assoluto Medio calcolato su tutto il test set:

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|$$

ed utilizzato solo per il dataset *Heart Failure Clinical Records* così da rendere il risultato interpretabile per un medico in termini di giorni effettivi di osservazione;

- **SA**: Scarto Assoluto:

$$\text{SA} = |y - \hat{y}|$$

per i primi 8 campioni del test set

Invece per l'analisi del comportamento dinamico dell'addestramento e la qualità delle predizioni sono state utilizzate due tipologie di rappresentazione grafica:

1. **Curva di apprendimento** che mostra la convergenza della funzione d'errore durante la fase di training;
2. **Confronto predizioni-valori reali** che valuta l'aderenza delle predizioni rispetto ai valori reali effettivi e la bontà del modello

Osservazione 4. Per facilitare la lettura visiva, i valori della funzione d'errore calcolati ad ogni epoca sono stati riportati in scala. In tal modo, l'andamento della convergenza viene espresso direttamente nell'unità di misura target (mg/m^3 o giorni).

3.1 Air Quality

Utilizzando la ReLU come funzione di attivazione e la regolarizzazione L2, la cross validation ha determinato come miglior configurazione:

`Strati nascosti = [16, 32]`

`Lambda = 0.1`

ottenendo il seguente risultato su 8 campioni del test set:

--- CONFRONTO PREDIZIONI-CAMPIONE (CO(GT)) ---

Campione	Valore reale	Valore predetto	Scarto	
1	1.80	1.55	0.25	mg/m ³
2	2.30	2.24	0.06	mg/m ³
3	1.60	2.03	0.43	mg/m ³
4	1.50	1.44	0.06	mg/m ³
5	1.70	1.53	0.17	mg/m ³
6	0.60	1.15	0.55	mg/m ³
7	7.20	5.92	1.28	mg/m ³
8	0.50	0.84	0.34	mg/m ³

Si mostrano ora i seguenti grafici ottenuti:

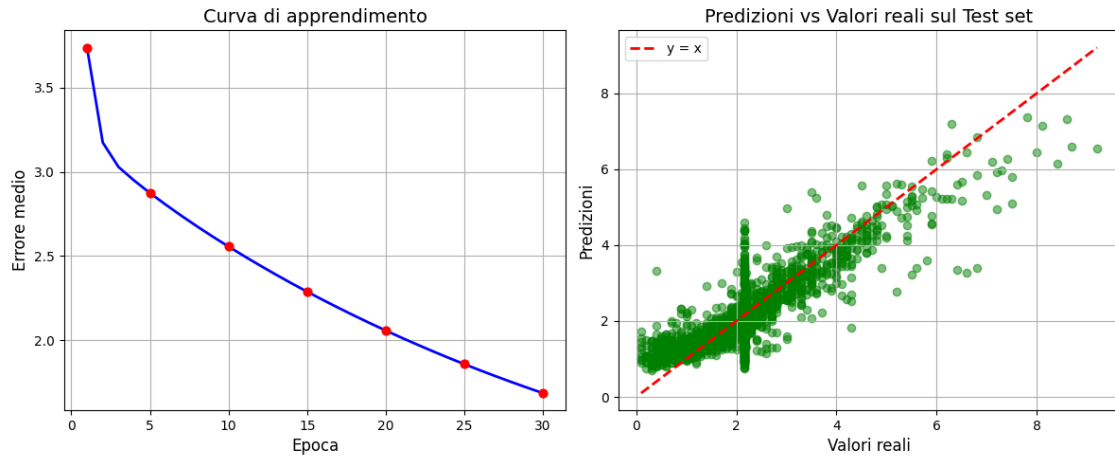


Figura 1.

Utilizzando invece la Tangente Iperbolica come funzione di attivazione e la regolarizzazione L2, la cross validation ha determinato come miglior configurazione:

`Strati nascosti = [16, 32]`

`Lambda = 0.1`

con i seguenti grafici

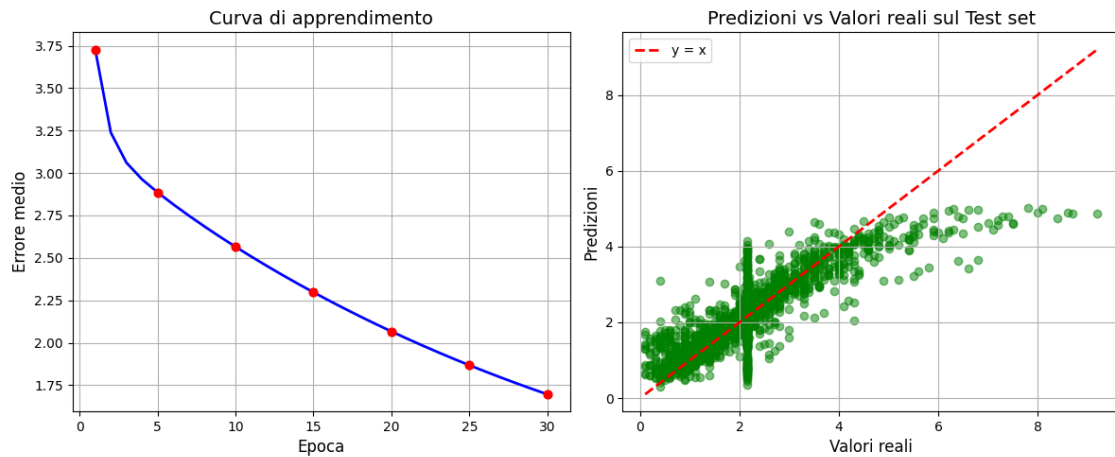


Figura 2.

In particolare si nota come la Tangente Iperbolica introduca un limite di saturazione in presenza di picchi di inquinamento ($>6m g/m^3$) si ha un upper bound oltre il quale le predizioni non riescono a salire.

Infatti la Tangente Iperbolica per definizione prende un valore z e lo schiaccia nell'intervallo $(-1, 1)$, ma per valori di inquinamento molto elevati la funzione non riesce più a distinguerli; inoltre la corrispettiva derivata diventa zero annullando di conseguenza l'aggiornamento dei pesi.

La ReLU invece non presenta un upper bound, dimostrando così che in tal caso quest'ultima risulta essere la funzione di attivazione migliore.

3.2 Heart Failure Clinical Records

Utilizzando la ReLU come funzione di attivazione e la regolarizzazione L2, la cross validation ha determinato come miglior configurazione:

```
Strati nascosti = [12, 16]
```

```
Lambda = 0.1
```

ottenendo il seguente risultato su 8 campioni del test set:

--- CONFRONTO PREDIZIONI-CAMPIONE (Tempo di sopravvivenza) ---

Campione	Valore reale	Valore predetto	Scarto
1	186.00	99.30	86.70 giorni
2	121.00	160.11	39.11 giorni
3	33.00	152.18	119.18 giorni
4	196.00	118.27	77.73 giorni
5	112.00	132.25	20.25 giorni
6	174.00	154.36	19.64 giorni
7	198.00	60.81	137.19 giorni
8	214.00	209.46	4.54 giorni

Media dei giorni di scarto: 66.86 giorni

Si mostrano ora i seguenti grafici ottenuti:

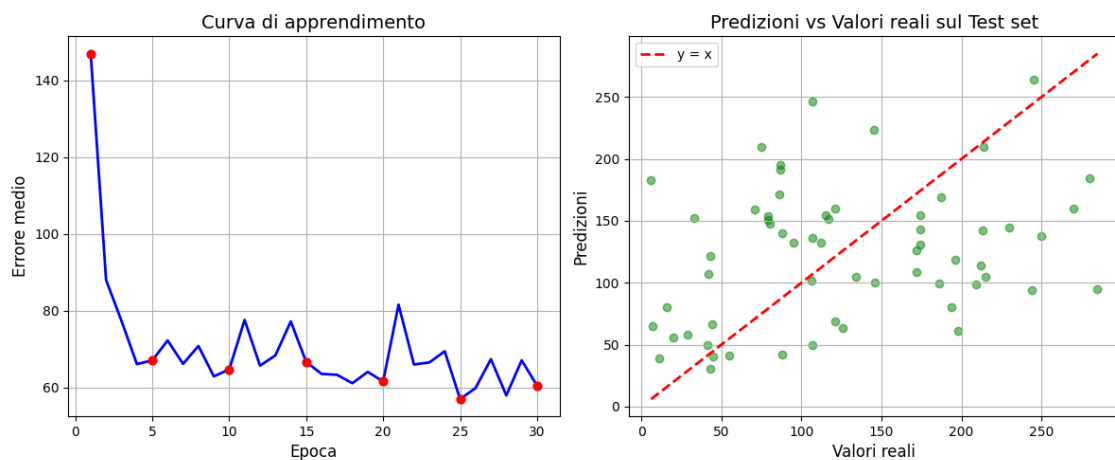


Figura 3.

In tal caso le oscillazioni dovrebbero essere causate dall'addestramento mini-batch SGD applicato ad un dataset di dimensioni ridotte

4 Istruzioni d'uso

Tale progetto è stato implementato in linguaggio Python e, come da specifica, non sono state utilizzate librerie di ottimizzazione ma solo quelle relative alla gestione dei dati.

Prima di eseguire il codice è necessario aver installato l'interprete Python e le relative dipendenze:

- MacOS: `pip3 install numpy pandas matplotlib`

- Windows: `pip install numpy pandas matplotlib`

Per avviare il programma, eseguire da terminale il file principale `main.py` come segue, a seconda del sistema operativo adottato:

- MacOS: digitare `python3 main.py`
- Windows: digitare `python main.py` oppure `py main.py`

All'avvio, il programma guida l'utente attraverso un menu interattivo:

1. Scelta del dataset:
 - digitare 1 per il dataset *Air Quality*
 - digitare 2 per il dataset *Heart Failure Clinical Records*
2. Scelta della funzione di attivazione:
 - digitare 1 per utilizzare la ReLU
 - digitare 2 per utilizzare la Tangente Iperbolica
3. Scelta del tipo di regolarizzazione:
 - digitare 1 per applicare la regolarizzazione L1
 - digitare 2 per applicare la regolarizzazione L2

Osservazione 5. Se si seleziona il dataset *Air Quality*, l'addestramento della rete neurale richiede un tempo di elaborazione superiore al dataset *Heart Failure Clinical Records* (circa 60 secondi), in quanto si ha un numero di feature e di mini-batch elevato.

5 Bibliografia

Per i riferimenti teorici sono state utilizzate le slide del corso e i relativi appunti presi dal sottoscritto, presenti nel file `Metodi di Ottimizzazione per Big Data.pdf`